AI game playing agents

NADA MOHAMED ALI 240481
MARIAM MOHAMMED SAYED 240434
JANA AMR MOHAMED 240120
MARIAM YASSER MOHAMMED 240435
YEHIA FARID YEHIA 240500
YOUSSEF WALID HAMDI 240523
YOUSSEF ISMAIL AHMED 240508
YOUSSEF AYMAN ABDELREHIM 240510

PEAS

Component	Description
Performance	Maximize wins, minimize losses, achieve higher win rate and efficient move selection
Environment	5×5 Tic-Tac-Toe board with two players taking alternating turns
Actuators	Place symbol (X or O) in an empty board cell
Sensors	Current board state, available legal moves, opponent actions

Algorithm Description

Rule-based agent

The algorithm implements a heuristic, rule-based decision-making process for a player (Agent) in a 5x5 Tic-Tac-Toe game. Unlike a standard 3x3 game, this version requires only **4 consecutive marks** (X or O) to win. The agent follows a strict hierarchy of priorities to determine the most "rational" move, ranging from immediate victory to random selection.

1.Game Environment and Rules:

Board: A 5 X 5 two-dimensional grid.

Winning Condition: Aligning 4 symbols horizontally, vertically, or diagonally.

Players: 'X' and 'O'.

Tie Condition: All 25 cells are filled without a winner.

2.The Heuristic Hierarchy (Priority Rules)

The core of the algorithm is the rule_based_agent function, which evaluates moves based on the following order of precedence. The agent executes the first rule that meets the criteria:

Win: If there is a move that results in an immediate victory (4 in a row), take it.

Block: If the opponent has a move that would lead to their victory on the next turn, occupy that spot to block them.

Fork: If a move creates two or more separate opportunities to win (two lines of 3 symbols that can be completed to 4), take that move.

Center: If the center cell (2, 2) is available, occupy it, as it offers the most strategic paths for diagonals and lines.

Corner: If a corner cell (0,0), (4,4), (0,4), or (4,0) is available, occupy it.

Random: If none of the above conditions are met, choose any available cell at random.

3. Functional Component:

Win Detection (checkWinner)

This function scans the board for any sequence of 4 identical symbols. It checks:

Rows: All horizontal segments of length 4.

Columns: All vertical segments of length 4.

Diagonals: Both primary diagonals and shorter diagonals adjacent to them (offset by 1).

Returns: 4 for 'X' win, -4 for 'O' win, 1 for a tie, and 0 if the game continues.

Defensive and Offensive Search (win_move & block_opponent)

win_move: This simulates every possible valid move for the current player. If a simulated move triggers the checkWinner function to return a win, that move is returned.

block_opponent: This identifies the opponent's symbol and uses the win_move logic from the *opponent's* perspective. If the opponent has a winning move, the agent intercepts it.

Fork Creation (fork_move)

A "Fork" is a strategic position where the player creates two potential winning paths simultaneously.

The algorithm simulates a move.

It then checks the "next state" to see if there are **two or more** different moves that would lead to a win.

If winning_chance >= 2, the move is identified as a fork.

Positional Strategy (center_move & corner_move)

These functions prioritize specific coordinates on the 5 X 5 grid:

Center: (2, 2) is the most valuable cell because it is the intersection of the most possible winning lines (horizontal, vertical, and both main diagonals).

Corners: (0,0), (0,4), (4,0), (4,4) are secondary priorities for establishing diagonal control.

4. Decision Flowchart (Logic Summary)

Input: Current board state and player symbol.

Scan for Win: Can I win now? (Yes → Move).

Scan for Threat: Can they win next? (Yes → Block).

Scan for Fork: Can I set up a double-threat? (Yes → Move).

Check Center: Is the middle free? (Yes → Move).

Check Corners: Are any corners free? (Yes → Move).

Default: Pick any random empty square.

Output: Chosen (row,column) coordinates.

Minimax agent

The algorithm implements a **Minimax** search strategy; a recursive decision-making algorithm used in two-player zero-sum games. The agent seeks to maximize its own score while assuming the opponent will play optimally to minimize it. To handle the large state space of a 5 X 5 board, it utilizes **Alpha-Beta Pruning** to eliminate branches that do not need to be explored and a **Heuristic Evaluation Function** to assess non-terminal game states.

Core Components

The Minimax Strategy (miniMax)

The algorithm explores the "game tree" up to a specified depth.

Maximizing Player (X): Attempts to choose the move that results in the highest possible score.

Minimizing Player (O): Attempts to choose the move that results in the lowest possible score.

Recursive Depth: The depth is limited (default = 3) to ensure the agent makes decisions within a reasonable timeframe, as a 5 X 5 board has trillions of possible states.

Alpha-Beta Pruning

This optimization technique reduces the number of nodes evaluated in the search tree:

Alpha: The best value that the maximizer is currently guaranteed.

Beta: The best value that the minimizer is currently guaranteed.

Pruning Condition: If at any point b <= a, the algorithm stops exploring that branch because the optimal player would never choose that path.

Heuristic Evaluation (evaluate & evaluateLine)

Since the agent cannot always search until the end of the game, it uses a heuristic to score the board:

Winning Lines: It scans all possible horizontal, vertical, and diagonal lines of length 4.

Scoring Weights: * **4-in-a-row:** +-10,000 points (Immediate win/loss).

- **3-in-a-row:** +- 100 points (Strong advantage).
- **2-in-a-row:** +- 10 points (Developing threat).

Blocking: If a line contains both 'X' and 'O', it is awarded 0 points because it can no longer be used for a win.

Decision Logic Flow

Generate Moves: The getBestMove function identifies all empty cells on the 5 X 5 board.

Simulate Play: For every possible move, the agent temporarily places its symbol and calls miniMax.

Recursive Evaluation:

- If a terminal state (Win/Loss/Tie) is reached, return a high static score.
- If the maximum depth is reached, return the score from the evaluate function.
- Pass the α and β values down the tree to prune unnecessary calculations.

Selection: The move that returns the highest score from the Minimax recursion is selected as the agent's final move.

Key Advantages

- **Forethought:** Unlike the Rule-Based agent, Minimax looks several turns ahead to anticipate opponent traps.
- **Efficiency:** Alpha-Beta pruning significantly speeds up the search, allowing for deeper "look-ahead" than a standard Minimax algorithm.

- **Flexibility:** By adjusting the depth or the evaluateLine weights, the agent's difficulty level can be easily tuned.

Q-Learning agent

The algorithm implements **Q-Learning**, a model-free Reinforcement Learning (RL) technique. The agent learns a policy by interacting with the environment, receiving rewards for good moves and penalties for bad ones. Over thousands of training episodes, it builds a **Q-Table** that maps game states to the expected long-term value of specific actions.

Reinforcement Learning Components

The Q-Table (Q)

State: The entire 5x5 board is converted into a string representation to serve as a unique key in a dictionary.

Action: Each possible cell (row, column) represents an action the agent can take.

Value: The Q-value represents the "quality" of a move. Higher values indicate moves more likely to lead to a win.

The Reward Function (get_reward)

Unlike simple Tic-Tac-Toe, this agent uses **Heuristic Rewards** to accelerate learning on a large 5x5 grid:

Terminal Rewards: Winning (+100), Losing (-100), or Drawing (+10).

Intermediate Rewards:

- **Move Penalty:** -1 for every move (encourages winning faster).
- **Three-in-a-Row:** +8 for creating a line of 3 symbols.
- **Blocking:** +5 for preventing the opponent from completing a line.

Exploration vs. Exploitation (choose_action)

The agent uses an **(Epsilon-Greedy)** strategy to balance learning and playing:

Exploration: Choosing a random move to discover new strategies. Initially, this is 100%.

Decay: The exploration rate decreases over time (multiplied by 0.995 each episode) until it reaches a minimum of 1\%

Exploitation: Once the agent is "trained," it ignores randomness and chooses the move with the highest Q-value from its table.

The Q-Learning Update Equation

For every move made during training, the agent updates its knowledge using the Bellman Equation:

Learning Rate (alpha = 0.1): Determines how much new information overrides old information.

Discount Factor (gamma = 0.9): Determines the importance of future rewards compared to immediate ones.

Max Future Q: The agent looks at the next state (s') and assumes it will pick the best possible move (a') thereafter.

Training Process

Initialize: Start with an empty Q-Table and high exploration.

Play Episodes: Run 50,000 games where two agents (or the agent against itself) play.

Observe & Update: After every move, calculate the reward and update the Q-value for that specific board state.

Save Knowledge: The final Q-Table is saved as a .pkl file (serialized data) so the agent can play against humans later without needing to re-learn.

Strategic Enhancements

Spatial Bias: During exploration, the agent is programmed to prefer moves within a distance of 2 from the center (2,2), helping it focus on the most strategically relevant area of the 5x5 board.

Win Detection: Uses a segment-scanning logic to find 4-in-a-row in any direction (horizontal, vertical, or diagonal).

Design Decisions

The project implements an intelligent multi-agent game environment for a 5×5 Tic-Tac-Toe game.

The system includes three different AI agents:

- Rule-Based Agent
- Minimax Agent with Alpha-Beta Pruning
- Q-Learning Agent

All agents interact with the same game environment and compete against each other through automated tournaments.

The purpose of the system is to compare different artificial intelligence techniques in terms of performance, efficiency, and decision-making quality.

System Architecture

The system is divided into several independent modules to improve maintainability, readability, and scalability.

Main system components:

1. Game Engine Design

The Game Engine is the core component of the system.

It manages:

- board representation
- player turns
- move validation
- winner detection
- draw conditions

The board is represented using a 5×5 matrix where:

- ' ' represents an empty cell
- 'X' represents Player X
- 'O' represents Player O.

Nested Python lists were used to represent the game board because they provide a simple and flexible way to store and manipulate two-dimensional data.

2. Rule-Based Agent Design

The Rule-Based Agent was designed using handcrafted heuristics.

The Rule-Based Agent was designed using handcrafted heuristics. The agent follows a fixed decision hierarchy:

1. ''Win immediately''
2. ''Block the opponent''
3. ''Create a fork opportunity''
4. ''Take the center position''
5. ''Take a corner position''
6. ''Select a random valid move''

This design was selected because it is computationally lightweight, easy to implement, and provides a baseline AI opponent for comparison with more advanced approaches.

3. Minimax Agent Design

The Minimax Agent was designed using adversarial search. The algorithm recursively evaluates possible future game states to determine the optimal move for the maximizing player.
To improve efficiency, Alpha–Beta pruning was integrated to eliminate branches that cannot influence the final decision, thereby reducing the number of nodes explored and computation time.
The search process uses a depth-limited strategy to control the exponential growth of the search space in the 5×5 board environment.

A heuristic evaluation function was implemented to estimate the value of non-terminal board states. This function assigns weighted scores based on potential winning lines of four cells, with higher weights given to stronger configurations such as three-in-a-row and four-in-a-row patterns for both offensive and defensive positions.

4. Q-Learning Agent Design

The Q-Learning Agent was designed using reinforcement learning principles. The agent stores state-action values in a Q-table, where each board state is mapped to action values representing expected long-term rewards. These values are updated iteratively through self-play using the Q-learning update rule.

Rewards are assigned based on game outcomes and intermediate game dynamics:
• a positive reward for winning
• a negative reward for losing
• a small positive reward for a draw
• a step penalty for each move to encourage faster wins
• additional heuristic rewards for creating three-in-a-row patterns and blocking opponent threats

The exploration rate gradually decreases over time, allowing the agent to shift from random exploration to learned exploitation. This enables the agent to progressively improve its strategy through repeated training episodes.

5. Tournament System Design

A tournament-based evaluation framework was developed to compare the performance of the Rule-Based, Minimax, and Q-Learning agents. Each agent plays against the others for a fixed number of games, with alternating starting positions to ensure fairness and reduce positional bias.

The game environment executes full matches by alternating turns between agents and evaluating outcomes using a win-detection function. Each match result is classified as a win, loss, or draw.

To provide a more quantitative comparison beyond win rates, an Elo rating system was implemented. Each agent starts with an initial rating of 1000, which is updated after every match based on expected performance versus actual outcomes. This allows dynamic tracking of relative agent strength over time.

The evaluation framework outputs both statistical performance metrics (win rate and draw rate) and ranking-based performance (Elo scores), enabling a comprehensive comparison of different AI strategies.

6.GUI Design

A graphical user interface (GUI) was designed to enable human interaction with the implemented AI agents. The interface presents the 5×5 game board visually and updates in real time after each move.

The GUI supports human vs AI gameplay by allowing the player to select board positions while the AI responds automatically using the selected agent strategy. It provides immediate visual feedback after each move, improving usability and interpretability of the game state.

The system displays the current board state after every action, enabling users to follow gameplay progression clearly from start to finish.

Results discussions

After finishing the implementation of the three agents, rule-based, minimax, and q-learning, we ran a tournament of 100 games between each two agents to compare the winning, losing and draw rates. In addition to calculating the ELO ratings.

When running the tournament function on minimax agent and rule-based agent , the minimax agent and rule-based agent achieved equal performance in this matchup, indicating comparable effectiveness in this specific configuration.

- Minimax Agent Wins: 50
- Rule-Based Agent Wins: 50
- Draws: 0

WIN RATES

- Minimax Agent Win Rate: 50.00%
- Rule-Based Agent Win Rate: 50.00%
- Draw Rate: 0.00%

ELO RATINGS

- Minimax Agent ELO: 1041
- Rule-Based Agent ELO: 1052

And when running the tournament function on the minimax agent and the q-learning agent, minimax slightly outperformed the q-learning agent, although the results remained relatively close:

- Minimax Agent Wins: 50
- Q-Learning Agent Wins: 49
- Draws: 1

WIN RATES

- Minimax Agent Win Rate: 50.00%
- Q-Learning Agent Win Rate: 49.00%

- Draw Rate: 1.00%

ELO RATINGS

- Minimax Agent ELO: 1037
- Q-Learning Agent ELO: 1048

While the last trial was on rule-based agent and the q-learning agent, the rule-based agent significantly outperformed the q-learning agent in this evaluation, demonstrating stronger decision-making in the tested training state of the model:

- Rule-Based Agent Wins: 91
- Q-Learning Agent Wins: 3
- Draws: 6

WIN RATES

- Rule-Based Agent Win Rate: 91.00%
- Q-Learning Agent Win Rate: 3.00%
- Draw Rate: 6.00%

ELO RATINGS

- Rule-Based Agent ELO: 1230
- Q-Learning Agent ELO: 765

The results indicate that the Minimax and Rule-Based agents perform at a relatively similar level in direct competition, while the Q-Learning agent shows weaker performance in its current training state. This suggests that the Q-Learning model may require additional training episodes or parameter tuning to achieve stronger generalization.

Overall, the Minimax agent demonstrates consistent performance due to its optimal search strategy, while the Rule-Based agent performs well in structured scenarios but lacks long-term planning capability.